Problem 1. Ford–Fulkerson / Edmonds–Karp (BFS; neighbors alphabetical)

Given: Directed network with source s and sink t with capacities (from the problem figure):

$$\begin{array}{lll} s \to a : 6 & s \to b : 1 & s \to c : 10 \\ a \to d : 4 & a \to b : 2 & a \to e : 1 \\ d \to g : 5 & d \to e : 2 & b \to e : 20 \\ c \to b : 2 & c \to f : 5 & e \to g : 10 \\ e \to f : 6 & g \to t : 12 & f \to t : 4 \end{array}$$

We run Edmonds–Karp (BFS for augmenting paths; when exploring a vertex, examine outgoing neighbors in alphabetical order). For each augmentation below we list: (i) the augmenting path found by BFS, (ii) the bottleneck and augmentation amount, and (iii) the residual graph G_f after the augmentation (only edges with positive residual capacity are listed; reverse edges created by flow are shown explicitly).

Step 1.
Augmenting path: $s \to c \to f \to t$.
Bottleneck: $\min\{c(s,c) = 10,\ c(c,f) = 5,\ c(f,t) = 4\} = 4$.
Augment by $+4$.

Residual graph G_f after Step 1 (nonzero residual capacities):

Forward edges (remaining capacity):
$s \to a : 6, \quad s \to b : 1, \quad s \to c : 6$
$a \to d : 4,\ a \to b : 2,\ a \to e : 1$
$d \to g : 5,\ d \to e : 2,\ b \to e : 20$
$c \to b : 2,\ c \to f : 1,\ e \to g : 10$
$e \to f : 6,\ g \to t : 12,\ f \to t : 0$

Reverse edges:
$c \to s : 4, \quad f \to c : 4, \quad t \to f : 4$

Step 2.
Augmenting path: $s \to a \to d \to g \to t$.
Bottleneck: $\min\{6, 4, 5, 12\} = 4$.
Augment by $+4$.

Residual graph G_f after Step 2:

Forward edges (remaining capacity):
$s \to a : 2, \quad s \to b : 1, \quad s \to c : 6$
$a \to d : 0,\ a \to b : 2,\ a \to e : 1$
$d \to g : 1,\ d \to e : 2,\ b \to e : 20$
$c \to b : 2,\ c \to f : 1,\ e \to g : 10$
$e \to f : 6,\ g \to t : 8,\ f \to t : 0$

Reverse edges (cumulative):
$a \to s : 4, \quad d \to a : 4, \quad g \to d : 4, \quad t \to g : 4$
$c \to s : 4, \quad f \to c : 4, \quad t \to f : 4$

Step 3.
Augmenting path: $s \to a \to e \to g \to t$.
Bottleneck: $\min\{c(s,a) = 2,\ c(a,e) = 1,\ c(e,g) = 10,\ c(g,t) = 8\} = 1$.
Augment by $+1$.

Residual graph G_f after Step 3:

Forward edges (remaining capacity):
$s \to a : 1, \quad s \to b : 1, \quad s \to c : 6$
$a \to d : 0,\ a \to b : 2,\ a \to e : 0$
$d \to g : 1,\ d \to e : 2,\ b \to e : 20$
$c \to b : 2,\ c \to f : 1,\ e \to g : 9$
$e \to f : 6,\ g \to t : 7,\ f \to t : 0$

Reverse edges (cumulative):
$a \to s : 5, \quad e \to a : 1, \quad g \to e : 1, \quad t \to g : 5$
$d \to a : 4, \quad g \to d : 4$
$c \to s : 4, \quad f \to c : 4, \quad t \to f : 4$

Step 4.
Augmenting path: $s \to b \to e \to g \to t$.
Bottleneck: $\min\{c(s,b) = 1,\ c(b,e) = 20,\ c(e,g) = 9,\ c(g,t) = 7\} = 1$.
Augment by $+1$.

Residual graph G_f after Step 4:

Forward edges (remaining capacity):
$s \to a : 1, \quad s \to b : 0, \quad s \to c : 6$
$a \to d : 0,\ a \to b : 2,\ a \to e : 0$
$d \to g : 1,\ d \to e : 2,\ b \to e : 19$
$c \to b : 2,\ c \to f : 1,\ e \to g : 8$
$e \to f : 6,\ g \to t : 6,\ f \to t : 0$

Reverse edges (cumulative):
$b \to s : 1, \quad e \to b : 1, \quad g \to e : 2, \quad t \to g : 6$
$a \to s : 5, \quad e \to a : 1$
$d \to a : 4, \quad g \to d : 4$
$c \to s : 4, \quad f \to c : 4, \quad t \to f : 4$

Step 5.
Augmenting path: $s \to a \to b \to e \to g \to t$.
Check residual capacities: $c(s,a) = 1,\ c(a,b) = 2,\ c(b,e) = 19,\ c(e,g) = 8,\ c(g,t) = 6$.
Bottleneck: 1.
Augment by $+1$.

Residual graph G_f after Step 5:

Forward edges (remaining capacity):
$s \to a : 0, \quad s \to b : 0, \quad s \to c : 6$
$a \to d : 0,\ a \to b : 1,\ a \to e : 0$
$d \to g : 1,\ d \to e : 2,\ b \to e : 18$
$c \to b : 2,\ c \to f : 1,\ e \to g : 7$
$e \to f : 6,\ g \to t : 5,\ f \to t : 0$

Reverse edges (cumulative):
$a \to s : 6, \quad b \to a : 1, \quad e \to b : 2, \quad g \to e : 3, \quad t \to g : 7$
$b \to s : 1, \quad e \to a : 1$
$d \to a : 4, \quad g \to d : 4$
$c \to s : 4, \quad f \to c : 4, \quad t \to f : 4$

Step 6.
Augmenting path: $s \to c \to b \to e \to g \to t$.
Residual capacities along path: $c(s,c) = 6,\ c(c,b) = 2,\ c(b,e) = 18,\ c(e,g) = 7,\ c(g,t) = 5$.
Bottleneck: $\min\{6, 2, 18, 7, 5\} = 2$.
Augment by $+2$.

Residual graph G_f after Step 6 (final residual graph):

Forward edges (remaining capacity):
$s \to a : 0, \quad s \to b : 0, \quad s \to c : 4$
$a \to d : 0,\ a \to b : 1,\ a \to e : 0$
$d \to g : 1,\ d \to e : 2,\ b \to e : 16$
$c \to b : 0,\ c \to f : 1,\ e \to g : 5$
$e \to f : 6,\ g \to t : 3,\ f \to t : 0$

Reverse edges (final cumulative list):
$a \to s : 6, \quad b \to a : 1, \quad b \to s : 1, \quad b \to c : 2$
$c \to s : 6, \quad d \to a : 4, \quad e \to a : 1, \quad e \to b : 4$
$f \to c : 4, \quad g \to d : 4, \quad g \to e : 5$
$t \to f : 4, \quad t \to g : 9$

Total flow: Summing augmentations $4 + 4 + 1 + 1 + 1 + 2 = 13$. Therefore the maximum s–t flow found is

$$\boxed{\text{max flow } = 13.}$$

Final flows on original edges (flow/capacity):

$s \to a : 6/6, \quad s \to b : 1/1, \quad s \to c : 6/10,$
$a \to d : 4/4, \quad a \to b : 1/2, \quad a \to e : 1/1,$
$d \to g : 4/5, \quad d \to e : 0/2, \quad b \to e : 4/20,$
$c \to b : 2/2, \quad c \to f : 4/5, \quad e \to g : 5/10,$
$e \to f : 0/6, \quad f \to t : 4/4, \quad g \to t : 9/12$

Minimum cut: Compute reachability from s in the final residual graph (edges with positive residual capacity). Reachable set

$$S = \{s, c, f\}, \qquad T = \{a, b, d, e, g, t\}.$$

Edges from S to T in the original graph are $s \to a$ (6), $s \to b$ (1), $c \to b$ (2), and $f \to t$ (4). Their capacities sum to $6 + 1 + 2 + 4 = 13$, equal to the max flow; hence (S, T) is a minimum cut and min-cut capacity $= 13$.

Problem 2. Max Flow with Vertex Capacities (reduction)

Goal: Reduce a flow problem in which every vertex v has a capacity $\kappa(v)$ (limit on total flow through v) to a standard max-flow problem having only edge capacities.

Reduction (vertex-splitting):

1. For each original vertex v create two vertices v_{in} and v_{out}.
2. Add an edge $v_{\text{in}} \to v_{\text{out}}$ with capacity $\kappa(v)$ (the vertex capacity).
3. For every original directed edge $u \to v$ with capacity $c(u, v)$, add an edge $u_{\text{out}} \to v_{\text{in}}$ with capacity $c(u, v)$.
4. Use s_{out} as the source side and t_{in} as the sink side (split s, t as well if they have vertex caps).

Correctness (sketch): Any flow in the transformed graph respects the $v_{\text{in}} \to v_{\text{out}}$ capacity, so the total net flow that can pass through v is at most $\kappa(v)$. Conversely, any feasible flow in the original graph with vertex capacities corresponds to a feasible flow in the transformed graph by routing the same amounts through the $v_{\text{in}} \to v_{\text{out}}$ edges. Thus the set of feasible flows and the maximum achievable s–t flow values correspond.

Size and complexity: If the original graph has n vertices and m edges, the transformed graph has $2n$ vertices and $m + n$ edges (one extra edge per original vertex plus one edge per original edge). Running any standard max-flow algorithm on the transformed graph therefore solves the vertex-capacity problem; the running time is that of the chosen algorithm on the transformed graph (e.g., Dinic, push–relabel, Edmonds–Karp on sizes $V' = 2n$, $E' = m + n$).

Problem 3. Critical Edge (find one efficiently)

Definition: An edge is *critical* if decreasing its capacity by some positive amount strictly decreases the maximum s–t flow.

Algorithm:

1. Compute a maximum s–t flow f in the network (any algorithm).
2. Construct the residual graph G_f for f and perform a reachability (BFS/DFS) from s in G_f. Let S be the set of reachable vertices and $T = V \setminus S$.
3. Any original edge $(u \to v)$ with $u \in S$ and $v \in T$ (i.e., crossing the cut (S, T)) is saturated in f and lies on a minimum cut. Return any such edge, it is critical.

Correctness: By the max-flow/min-cut theorem the capacity of the (S, T) cut equals the maximum flow. Any edge from S to T contributes to the cut capacity; decreasing its capacity decreases the cut capacity and therefore strictly decreases the maximum possible flow. Such an edge is thus critical.

Running time: One max-flow computation plus one BFS/DFS in the residual graph. So the time is dominated by the chosen max-flow algorithm; the extra step is $O(V + E)$.

Problem 4. Graduation Rules — flow formulation and decision algorithm

Given: Course set $C = \{c_1, \ldots, c_n\}$. Rules $(S_1, x_1), \ldots, (S_m, x_m)$ where rule i requires at least x_i courses from subset $S_i \subseteq C$. Each course may be used to satisfy at most one rule. A student has taken a set of ℓ courses $T \subseteq C$. Decide whether the student can graduate (i.e., assign taken courses to rules so each rule i receives at least x_i distinct courses from S_i).

Flow network construction:

- Source s.
- For each taken course $c_j \in T$ create a node C_j and edge $s \to C_j$ with capacity 1.
- For each rule i create a node R_i and edge $R_i \to t$ with capacity x_i.
- For each taken course $c_j \in T$ and each rule i with $c_j \in S_i$, add edge $C_j \to R_i$ with capacity 1.
- Sink t.

Decision: Compute the max flow. Let $X = \sum_{i=1}^{m} x_i$. If the maximum flow value equals X, then there exists an assignment of taken courses to rules meeting all requirements and the student can graduate. Otherwise they cannot.

Correctness (sketch): Flow units from s to t correspond to assignments of one taken course to one rule. Each course node has capacity 1 from s, ensuring it is used at most once. Each rule node can receive at most x_i units and must receive exactly x_i units for the total flow to reach X. Therefore achieving total flow X is equivalent to satisfying all rules simultaneously.

Complexity: Building the network takes $O(\ell + \#\{(c_j, R_i) : c_j \in S_i\})$ time and edges. Running the chosen max-flow algorithm dominates the time (e.g., Dinic on this bipartite-style graph). This is polynomial-time and practical for typical instance sizes.